Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas
Universidad Complutense de Madrid
Óscar Martín — Curso 2026/27

21. Texto

- Recuerda: `type String = [Char]`.

Funciones predefinidas sobre `String`

- Se pueden usar todas las funciones sobre listas.
- `lines :: String -> [String]`
  - `lines "abc\nde" ↝ ["abc", "de"]`
- `unlines :: [String] -> String`
  - `unlines ["abc", "de"] ↝ "abc\nde\n"`
- `words :: String -> [String]`
  - `words "abc de\nfgh" ↝ ["abc", "de", "fgh"]`
- `unwords :: [String] -> String`
  - `unwords ["abc", "de"] ↝ "abc de"`
- Existe `Data.Char`.

`show` y `read`

- `show :: Show a => a -> String`
  - `show 3 ↝ "3"`
- `read :: Read a => String -> a`
  - El tipo del resultado se infiere del contexto: `read "3" + 1.0`.
  - O con anotación explícita: `read "44" :: Int` o `read "44" :: Float`.
- Se espera que `show` produzca algo que Haskell pueda leer, de modo que `read (show x)` sea `x`.
  - Haskell no lo comprueba: es un convenio.
  - Un `show` *correcto*:

```haskell
data Intervalo = Intervalo Char Char
               deriving Read

instance Show Intervalo where
    show (Intervalo x y) =
        "Intervalo " ++ show x ++ " " ++ show y
```

  * `show (Intervalo 'a' 'z')` ↝ `"Intervalo 'a' 'z'"`
  * `read (show (Intervalo 'a' 'z')) :: Intervalo` ↝ `Intervalo 'a' 'z'`

- Un `show` *incorrecto*:

```haskell
data Intervalo' = Intervalo' Char Char

instance Show Intervalo' where
    show (Intervalo' x y) = [x] ++ ".." ++ [y]
```

  * `show (Intervalo' 'a' 'z')` ↝ `"a..z"`

Ejercicios

1. Escribe una función que compare dos cadenas de caracteres sin tener en cuenta los espacios en blanco: `sonIguales "ab cd" "abc  d"` ↝ `True`. Intenta conseguirlo usando la función `words`.

2. ¿Qué hacen `words . unwords` y `unwords . words`? (Recuerda que `.` es la composición de funciones.)

3. Se dice que el césar (algún césar) codificaba sus mensajes para que el enemigo no los pudiera entender aunque los interceptara. La codificación del césar consiste en sustituir cada letra por la que está un cierto número de posiciones por delante (o por detrás). Digamos que elegimos un movimiento de 3 posiciones, de manera que `'a'` se convierte en `'d'` y `"Hola"` se convierte en `"Krod"`. El movimiento de 3 posiciones es cíclico, de forma que `'x'` se convierte en `'a'`. Vamos a suponer que el mensaje original solo contiene letras, no espacios ni otros símbolos. Define esta codificación como una función Haskell. Puede resultar útil saber a qué clases pertenece `Char`.